



TECHNICAL PORTFOLIO

# Engineering Project Showcase

Production-Grade Systems & Developer Tools

Software Engineering Portfolio

June 2026



# Contents

---

## **Project 1: Production-Grade API Service**

System Architecture & Directory Layout

Key Specifications & Engineering Features

Core Engineering Proficiencies Demonstrated

## **Project 2: Data Pipeline / ETL System**

Pipeline Flow & Orchestration Topology

Key Specifications & Engineering Features

Analytical Database Target Schema

Core Engineering Proficiencies Demonstrated

## **Project 3: CLI Tool / Developer Productivity Tool**

Application Structure

Key Specifications & Engineering Features

Core Engineering Proficiencies Demonstrated

## PROJECT 1 — 2-3 WEEKS

# Production-Grade API Service

A high-concurrency, asynchronous Task Management and Execution API featuring multi-tenant isolation, structured data validation, and automated background workers.

## System Architecture & Directory Layout

```

src/
├── app/
│   ├── api/
│   │   └── v1/
│   │       ├── endpoints/
│   │       │   ├── auth.py          # JWT lifecycle, token refresh, Argon2 hashing
│   │       │   └── tasks.py         # Paginated CRUD, dynamic filtering & sorting
│   │       └── router.py
│   ├── core/
│   │   ├── config.py               # Pydantic BaseSettings environment management
│   │   └── security.py             # PyJWT encryption, asymmetric keys, scope
│   └── validation
│       ├── database.py             # Async SQLAlchemy 2.0 engine & session factory
│       ├── models/                 # Declarative database core models
│       │   ├── user.py
│       │   └── task.py
│       ├── schemas/                # Pydantic v2 strict serialization models
│       │   ├── user.py
│       │   └── task.py
│       ├── services/               # Encapsulated transactional business logic
│       │   └── task_service.py
│       └── worker/                 # Celery background worker application
│           ├── tasks.py            # Out-of-band digest & notification tasks
│           └── tasks.py
├── alembic/                        # Database migration timeline scripts
├── tests/                          # Comprehensive Pytest suite
│   ├── conftest.py                # Testcontainers for Postgres/Redis fixtures
│   └── test_api/
└── Dockerfile                      # Multi-stage production container build
  
```

## Key Specifications & Engineering Features

- **Advanced Authentication & RBAC:** Implements an OAuth2 Password Bearer workflow using asymmetric JSON Web Tokens (RS256). Granular access control is verified at the routing layer using explicit token scopes ( `scopes: ["user:read", "task:write"]` ).

- **Asynchronous Database Optimization:** Utilizes async SQLAlchemy 2.0 operations. Explicitly leverages `selectinload` for relationship fetching to eliminate the N+1 query problem under heavy load. Implements deterministic keyset pagination ( `cursor` -based) to maintain constant-time  $O(1)$  query overhead over millions of records.
- **Sliding-Window Rate Limiting:** Features custom Redis-backed middleware that intercepts requests prior to route execution. Uses a Sliding Window Counter algorithm via Redis sorted sets ( `ZSET` ) to throttle requests dynamically based on user authentication tiers.
- **Resilient Worker Offloading:** Integrates Celery with a Redis message broker to process long-running, resource-intensive operations (such as data exports and email notifications). Includes automated exponential backoff retries with execution jitter.
- **Production Deployment Rigor:** Packaged using a multi-stage `Dockerfile` pinning dependencies securely. Runs within a minimal `python:3.11-slim` image under a non-root system user to reduce security vulnerabilities. Includes a structured GitHub Actions workflow managing linting ( `ruff` ), type-checking ( `mypy` ), and continuous integration tests with coverage reporting ( `pytest-cov` ).

## Core Engineering Proficiencies Demonstrated

- Mastery of asynchronous coroutine patterns and non-blocking I/O in Python.
- Test-Driven Development (TDD) principles utilizing mock fixtures for stateful dependencies.
- Edge validation and data sanitization techniques to secure external interfaces.

## PROJECT 2 — 2-3 WEEKS

# Data Pipeline / ETL System

---

A high-throughput, fault-tolerant Geographical Climate Trends ETL Pipeline that extracts, standardizes, and validates unstructured weather streaming matrices into a structured time-series data warehouse.

## Pipeline Flow & Orchestration Topology

```
dags/
├─ weather_etl_dag.py           # Apache Airflow DAG orchestrating the pipeline
schedule
scripts/
├─ extract.py                 # Multi-threaded API consumer with circuit-breaker
backoffs
├─ transform.py              # Vectorized structural cleaning and normalization
via Pandas
├─ load.py                   # Bulk insert execution using SQLAlchemy Core batch
loading
config/
├─ great_expectations/      # Structural invariant testing suites & assertion
logic
├─ expectations/
└─ weather_batch_rules.json
```

## Key Specifications & Engineering Features

- **Robust Extraction Mechanics:** Executes multi-threaded data fetching against public weather APIs. Uses the `tenacity` library to build fault-tolerant request wrappers capable of handling transient 429 Too Many Requests or connection errors gracefully.
- **Vectorized Transformation Engine:** Utilizes Pandas to apply pure, deterministic transformations across extracted batches: flattens deeply nested JSON structures into uniform records; normalizes variable time zones into strict ISO 8601 UTC formats; implements forward-filling imputation algorithms to handle data gaps across neighboring geographical sectors within fixed time windows.
- **Idempotent Data Ingestion:** Built around an Upsert (Merge) database pattern in PostgreSQL. Raw data is loaded into an unindexed transient staging table before execute-

or-update queries run atomically within a transaction bounded by the Apache Airflow execution date ( `ds` macro). This prevents duplicate writes during historical backfills.

- **Automated Data Quality Gates:** Integrates Great Expectations into the Airflow DAG chain immediately before final loading. Validates critical data constraints: `expect_column_values_to_not_be_null(column="station_id")` and `expect_column_values_to_be_between(column="temperature_celsius", min_value=-60, max_value=60)`. Isolates failed data batches to a quarantine table and sends instant alerting notifications via a webhook if structural failure bounds exceed 1%.

## Analytical Database Target Schema

```
CREATE TABLE fact_weather_observations (
  observation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  station_id VARCHAR(50) NOT NULL,
  observed_at TIMESTAMP WITH TIME ZONE NOT NULL,
  temperature_celsius NUMERIC(5, 2),
  humidity_percentage NUMERIC(5, 2),
  wind_speed_ms NUMERIC(5, 2),
  inserted_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
) PARTITION BY RANGE (observed_at);
```

## Core Engineering Proficiencies Demonstrated

- Design of deterministic, idempotent data workflows that safely support historical reruns.
- Optimization of analytical storage systems, including partitioning strategies and index tuning.
- Real-time pipeline monitoring, logging, and data-quality enforcement.

## PROJECT 3 — 1-2 WEEKS

# CLI Tool / Developer Productivity Tool

---

A highly performant, local developer utility named `git-radar` that interrogates repository lifecycles to detect structural code decay and development anomalies.

## Application Structure

```
git_radar/
├── __init__.py
├── __main__.py           # Application entry point mapping terminal commands
├── cli.py                # Typer UI bindings, command contextual
initialization
├── analyzer.py          # High-speed git commit graph traversal engine
├── exporters.py          # Structured data encoders for JSON, CSV, and HTML
reports
├── config.py             # YAML parser utilizing Pydantic validation
pyproject.toml           # Modern Poetry metadata, build-system, and package
settings
```

## Key Specifications & Engineering Features

- **Ergonomic Terminal Interfaces:** Uses `Typer` to build clean, intuitive command-line interfaces complete with argument checking and shell autocompletion. Integrates `Rich` to handle responsive UI elements, rendering colored analytics dashboards and real-time operations inside the terminal.
- **Deep Code Tree Interrogation:** Interfaces natively with local files using `GitPython` to traverse repository commit logs. Analyzes complex file changes over time to compute critical team metrics: **Code Hotspots** maps file paths by mutation frequency versus test coverage growth to flag high-risk areas; **Bus Factor Calculations** identifies developer ownership concentration metrics across specific sub-directories.
- **External Configuration Management:** Leverages a local `.radar.yaml` file to parse and enforce repository exclude paths, author aliasing rules, and quality thresholds.
- **Automated Package Delivery:** Conforms strictly to modern PEP standards via a modular `pyproject.toml` layout managed by `poetry-core`. Features a completely hands-off

continuous deployment loop via GitHub Actions that tests, builds, and publishes semantic releases directly to PyPI when release tags are pushed.

## Core Engineering Proficiencies Demonstrated

- Building clean, reliable command-line interfaces that simplify complex back-end data tasks.
- Deep knowledge of internal version control graphs and repository structural tracking.
- Standardized Python package management, distribution patterns, and automated CI/CD pipelines.